

BABU BANARASI DAS

UNIVERSITY



2026-2027

**SCHOOL OF COMPUTER
APPLICATIONS**

No SQL Project On

Student Management System Using

MongoDB

Submitted to:

Mr. Harendra Singh

Submitted by:

RAVISH KUMAR

Department: BCA (DS & AI)

Roll no: (36)

Date: 25th September
no

university roll :
1250258351

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher Mr. Harendra Singh for giving me the opportunity to work on this NoSQL project titled "Student Management System Using MongoDB".

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to create, retrieve, update, Sorting and delete data. I also learned how different query operators such as greater than, less than, equal to, AND, and OR are used in MongoDB.

I am thankful to my teacher for the guidance and support provided throughout the completion of this project.

INTRODUCTION

MongoDB is a popular NoSQL database management system that stores data in the form of documents. Unlike traditional relational databases, MongoDB does not require data to be stored in fixed rows and columns.

In this project, a Student Management System has been created using MongoDB. The database contains information about 50 students, including their roll number, name, age, marks, and city.

Different MongoDB operations have been performed on the student data, such as inserting, displaying, updating, deleting, sorting, counting, and searching records. Comparison operators such as \$eq, \$gt, \$lt, \$gte, and \$lte have also been demonstrated.

OBJECTIVES

To understand the concept of NoSQL databases.

To learn the basic features of MongoDB.

To create a database and collection.

To insert 50 student records.

To perform CRUD operations.

To use comparison operators.

To perform sorting and filtering.

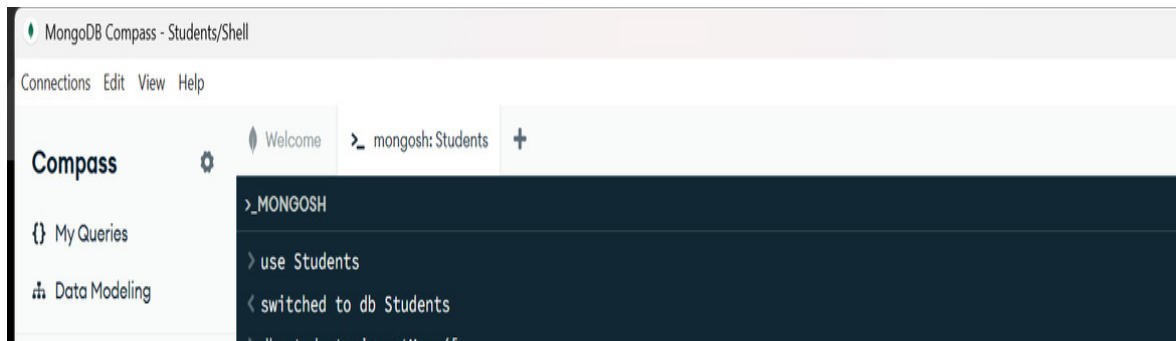
To understand AND and OR operations.

1. CREATE DATABASE

MongoDB Query:

use Students

Explanation: Creates or switches to the Student database.

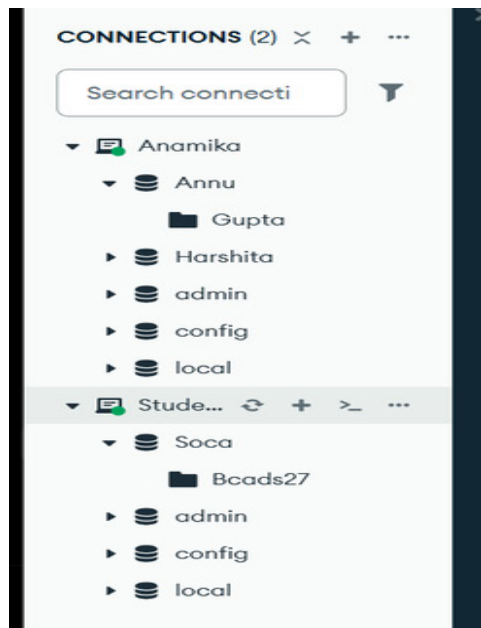


2. CREATE COLLECTION

MongoDB Query:

db.create Collection("students")

Explanation: Creates a collection named students.



3. INSERT 50 STUDENTS

MongoDB Query:

db. students. Insert Many([])

{roll:1, name:"Anamika", age:19,marks:88,city:"Lucknow"},

{roll:2, name:"Uday", age:18,marks:76,city:"Jaunpur"},

{roll:3, name:"Vaishnavi", age:19,marks:92,city:"Kanpur"},

{roll:4, name:"Annu", age:18,marks:69,city:"Ayodhya"},

{roll:5, name:"Vansh", age:19,marks:81,city:"Varanasi"},

{roll:6, name:"Amrita", age:20,marks:85,city:"Lucknow"},

{roll:7, name:"Naveen", age:19,marks:73,city:"Prayagraj"},

{roll:8, name:"Pratik", age:19,marks:90,city:"Lucknow"},

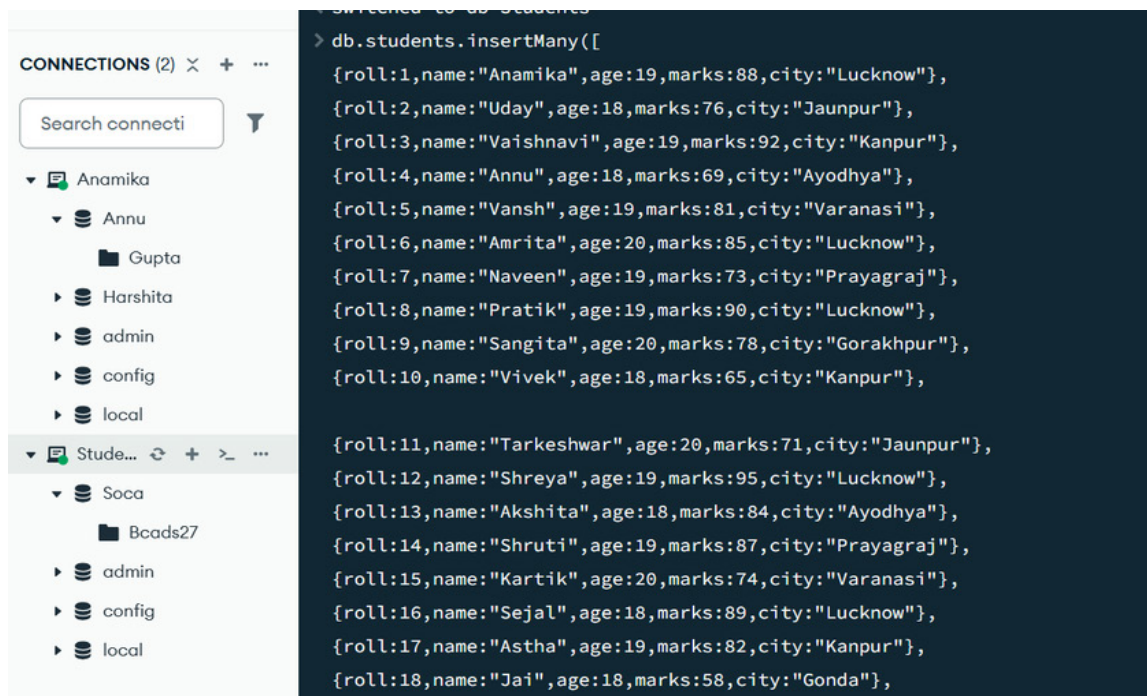
{roll:9, name:"Sangita", age:20,marks:78,city:"Gorakhpur"},

{roll:10, name:"Vivek", age:18,marks:65,city:"Kanpur"},

{roll:11, name:"Tarkeshwar", age:20,marks:71,city:"Jaunpur"},

{roll:12, name:"Shreya", age:19,marks:95,city:"Lucknow"},
{roll:13, name:"Akshita", age:18,marks:84,city:"Ayodhya"},
{roll:14, name:"Shruti", age:19,marks:87,city:"Prayagraj"},
{roll:15, name:"Kartik", age:20,marks:74,city:"Varanasi"},
{roll:16, name:"Sejal", age:18,marks:89,city:"Lucknow"},
{roll:17, name:"Astha", age:19,marks:82,city:"Kanpur"},
{roll:18, name:"Jai", age:18,marks:58,city:"Gonda"},
{roll:19, name:"Harshita", age:20,marks:91,city:"Lucknow"},
{roll:20, name:"Omkar", age:19,marks:63,city:"Bareilly"},
{roll:21, name:"Ayush", age:18,marks:77,city:"Lucknow"},
{roll:22, name:"Saumya", age:19,marks:86,city:"Jaunpur"},
{roll:23, name:"Vidya", age:20,marks:72,city:"Kanpur"},
{roll:24, name: "Adarsh", age:19,marks:80,city:"Ayodhya"},
{roll:25, name: "Anmol", age:18,marks:67,city:"Prayagraj"},
{roll:26, name: "Anushka", age:20,marks:94,city:"Lucknow"},
{roll:27, name: "Ananya", age:19,marks:88,city:"Varanasi"},
{roll:28, name: "Bhaskar", age:18,marks:61,city:"Gorakhpur"},
{roll:29, name: "Rohan", age:19,marks:75,city:"Kanpur"},
{roll:30, name: "Nishant", age:20,marks:79,city:"Lucknow"},
{roll:31, name: "Vineet", age:18,marks:68,city:"Jaunpur"},
{roll:32, name: "Anika", age:19,marks:90,city:"Lucknow"},
{roll:33, name: "Jai", age:20,marks:55,city:"Ayodhya"},
{roll:34, name: "Lakshya", age:18,marks:83,city:"Kanpur"},
{roll:35, name: "Krishna", age:19,marks:96,city:"Prayagraj"},
{roll:36, name: "Rumana", age:20,marks:70,city:"Lucknow"},

{roll:37, name: "Priyanshu", age:18,marks:81,city:"Varanasi"},
 {roll:38, name: "Aditya", age:19,marks:66,city:"Gorakhpur"},
 {roll:39, name: "Dhruv", age:20,marks:93,city:"Lucknow"},
 {roll:40, name: "Nivedita", age:19,marks:85,city:"Kanpur"},
 {roll:41, name: "Meenu", age:18,marks:74,city:"Jaunpur"},
 {roll:42, name: "Akanksha", age:20,marks:89,city:"Lucknow"},
 {roll:43, name: "Sarika", age:19,marks:62,city:"Ayodhya"},
 {roll:44, name: "Anchal", age:18,marks:78,city:"Prayagraj"},
 {roll:45, name: "Umra", age:20,marks:84,city:"Lucknow"},
 {roll:46, name: "Nashra", age:19,marks:73,city:"Kanpur"},
 {roll:47, name: "Aman", age:18,marks:59,city:"Gonda"},
 {roll:48, name: "Priya", age:20,marks:87,city:"Lucknow"},
 {roll:49, name: "Rahul", age:19,marks:64,city:"Bareilly"},
 {roll:50, name: "Neha", age:18,marks:91,city:"Varanasi"}



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS (2)' panel is visible, showing a search bar and a list of connections. The 'Stude...' connection is selected, showing a tree view with 'Soca' and 'Bcads27' collections. The 'Bcads27' collection is expanded, showing a list of documents. The main panel displays the 'db.students.insertMany()' command, which inserts 18 documents into the 'students' collection. Each document contains 'roll', 'name', 'age', 'marks', and 'city' fields.

```

> db.students.insertMany([
  {roll:1,name:"Anamika",age:19,marks:88,city:"Lucknow"},
  {roll:2,name:"Uday",age:18,marks:76,city:"Jaunpur"},
  {roll:3,name:"Vaishnavi",age:19,marks:92,city:"Kanpur"},
  {roll:4,name:"Annu",age:18,marks:69,city:"Ayodhya"},
  {roll:5,name:"Vansh",age:19,marks:81,city:"Varanasi"},
  {roll:6,name:"Amrita",age:20,marks:85,city:"Lucknow"},
  {roll:7,name:"Naveen",age:19,marks:73,city:"Prayagraj"},
  {roll:8,name:"Pratik",age:19,marks:90,city:"Lucknow"},
  {roll:9,name:"Sangita",age:20,marks:78,city:"Gorakhpur"},
  {roll:10,name:"Vivek",age:18,marks:65,city:"Kanpur"},

  {roll:11,name:"Tarkeshwar",age:20,marks:71,city:"Jaunpur"},
  {roll:12,name:"Shreya",age:19,marks:95,city:"Lucknow"},
  {roll:13,name:"Akshita",age:18,marks:84,city:"Ayodhya"},
  {roll:14,name:"Shruti",age:19,marks:87,city:"Prayagraj"},
  {roll:15,name:"Kartik",age:20,marks:74,city:"Varanasi"},
  {roll:16,name:"Sejal",age:18,marks:89,city:"Lucknow"},
  {roll:17,name:"Astha",age:19,marks:82,city:"Kanpur"},
  {roll:18,name:"Jai",age:18,marks:58,city:"Gonda"},
])
  
```

Explanation: Inserts multiple student documents into the collection.

4. DISPLAY ALL STUDENTS

MongoDB Query:

db. students. Find ()

Explanation: Retrieves documents from the students collection.

```
> db.students.find()
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221d'),
  roll: 2,
  name: 'Uday',
  age: 18,
  marks: 76,
  city: 'Jaunpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
```

5. EQUAL TO — \$eq

MongoDB Query:

db. students. Find ({marks: {\$eq: 88}})

Explanation: Finds students whose marks are exactly 88.

```
> db.students.find({marks: {$eq: 88}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2236'),
  roll: 27,
  name: 'Ananya',
  age: 19,
  marks: 88,
  city: 'Varanasi'
}
```

6. GREATER THAN — \$gt

MongoDB Query:

```
db.students.find({marks: {$gt: 85}})
```

Explanation: Finds students whose marks are greater than 85.

```
> db.students.find({marks: {$gt: 85}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
  marks: 92,
  city: 'Kanpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2223'),
  roll: 8,
  name: 'Pratik',
  age: 19,
  marks: 90,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2227'),
  roll: 12,
  name: 'Shreya',
  age: 19,
  marks: 95,
  city: 'Lucknow'
}
```

7. LESS THAN — \$lt

MongoDB Query:

db.students.find({marks: {\$lt: 60}})

Explanation: Finds students whose marks are less than 60.

```
> db.students.find({marks: {$lt: 60}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df222d'),
  roll: 18,
  name: 'Jai',
  age: 18,
  marks: 58,
  city: 'Gonda'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df223c'),
  roll: 33,
  name: 'Jai',
  age: 20,
  marks: 55,
  city: 'Ayodhya'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df224a'),
  roll: 47,
  name: 'Aman',
  age: 18,
  marks: 59,
  city: 'Gonda'
}
```

8. GREATER THAN OR EQUAL TO — \$gte

MongoDB Query:

db.students.find({marks: {\$gte: 90}})

Explanation: Finds students whose marks are 90 or above.

```
> db.students.find({marks: {$gte: 90}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
  marks: 92,
  city: 'Kanpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2223'),
  roll: 8,
  name: 'Pratik',
  age: 19,
  marks: 90,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2227'),
  roll: 12,
  name: 'Shreya',
  age: 19,
  marks: 95,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df222e'),
  roll: 19,
  name: 'Harshita',
  age: 20,
  marks: 91,
  city: 'Lucknow'
}
```

9. LESS THAN OR EQUAL TO — \$lte

MongoDB Query:

db.students.find({marks: {\$lte: 70}})

Explanation: Finds students whose marks are 70 or below.

```
> db.students.find({marks: {$lte: 70}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221f'),
  roll: 4,
  name: 'Annu',
  age: 18,
  marks: 69,
  city: 'Ayodhya'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2225'),
  roll: 10,
  name: 'Vivek',
  age: 18,
  marks: 65,
  city: 'Kanpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df222d'),
  roll: 18,
  name: 'Jai',
  age: 18,
  marks: 58,
  city: 'Gonda'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df222f'),
  roll: 20,
  name: 'Omkar',
  age: 19,
  marks: 63,
  city: 'Bareilly'
}
```

10. UPDATE OPERATION

MongoDB Query:

```
db.students.updateOne( {roll:5}, {$set:{marks:95}})
```

Explanation: Updates the marks to 95 of the student with roll number 5.

```
> db.students.updateOne(
  {roll:5},
  {$set:{marks:95}}
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.students.find({roll:5})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df2220'),
  roll: 5,
  name: 'Vansh',
  age: 19,
  marks: 95,
  city: 'Varanasi'
```

11. DELETE ONE OPERATION

MongoDB Query:

```
db.students.deleteOne({roll:10})
```

Explanation: Deletes one document matching roll number 10.

```
}
> db.students.deleteOne({roll:10})
< {
  acknowledged: true,
  deletedCount: 1
}
> db.students.find({roll:10})
<
Students >
```

12. DELETE MANY OPERATION

MongoDB Query:

```
db.students.deleteMany({marks: {$lt:60}})
```

Explanation: Deletes all students having marks below 60.

```
<
> db.students.deleteMany({marks:{$lt:60}})
< {
  acknowledged: true,
  deletedCount: 3
}
> db.students.find({marks:{$lt:60}})
<
Students > |
```

13. AND OPERATION — \$and

MongoDB Query:

```
db.students.find({ $and:[ {marks:{$gt:80}}, {age:19} ]})
```

Explanation: Finds students satisfying both conditions.

```
>_MONGOSH
> db.students.find({
  $and:[
    {marks:{$gt:80}}, {age:19}
  ]
})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
  marks: 92,
  city: 'Kanpur'
}
{
```

14. OR OPERATION — \$or

MongoDB Query:

```
db.students.find({ $or:[ {city:"Lucknow"}, {city:"Kanpur"} ]})
```

Explanation: Finds students from Lucknow or Kanpur.

```
>_MONGOSH
> db.students.find({
  $or:[
    {city:"Lucknow"},{city:"Kanpur"}
  ]
})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
  marks: 92,
  city: 'Kanpur'
}
{
```

15. SORT OPERATION

A.MongoDB Query:

db. students. find (). sort({marks : 1})

Explanation: Sorts students by marks from lowest to highest. Use 1 for ascending order.

```
> _MONGOSH
> db.students.find().sort({marks:1})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df2237'),
  roll: 28,
  name: 'Bhaskar',
  age: 18,
  marks: 61,
  city: 'Gorakhpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2246'),
  roll: 43,
  name: 'Sarika',
  age: 19,
  marks: 62,
  city: 'Ayodhya'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df222f'),
  roll: 20,
  name: 'Omkar',
  age: 19,
  marks: 63,
  city: 'Bareilly'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df224c'),
  roll: 49,
  name: 'Rahul',
  age: 19,
  marks: 64,
  city: 'Bareilly'
}
```

B. MongoDB Query:

db.students.find().sort({marks:-1})

Explanation: Sorts students by marks from highest to lowest. Use -1 for descending order.

```
>_MONGOSH
> db.students.find().sort({marks:-1})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df223e'),
  roll: 35,
  name: 'Krishna',
  age: 19,
  marks: 96,
  city: 'Prayagraj'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2227'),
  roll: 12,
  name: 'Shreya',
  age: 19,
  marks: 95,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2220'),
  roll: 5,
  name: 'Vansh',
  age: 19,
  marks: 95,
  city: 'Varanasi'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2235'),
  roll: 26,
  name: 'Anushka',
  age: 20,
  marks: 94,
  city: 'Lucknow'
}
```

16. LIMIT OPERATION

MongoDB Query:

db. students. find (). limit (5)

Explanation: Displays only the first five documents.

```
>_MONGOSH
> db.students.find().limit(5)
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221d'),
  roll: 2,
  name: 'Uday',
  age: 18,
  marks: 76,
  city: 'Jaunpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
```

17. SEARCH BY CITY

MongoDB Query:

```
db.students.find({city:"Lucknow"})
```

Explanation: Finds all students whose city is Lucknow.

```
>_MONGOSH
> db.students.find({city:"Lucknow"})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2221'),
  roll: 6,
  name: 'Amrita',
  age: 20,
  marks: 85,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2223'),
  roll: 8,
  name: 'Pratik',
  age: 19,
```

18. COUNT DOCUMENTS

MongoDB Query:

db. students. count Documents ()

Explanation: Counts documents currently present in the collection (performing after the delete function).

```
>_MONGOSH
```

```
}
```

```
> db.students.countDocuments()
```

```
< 46
```

MONGODB OPERATORS USED:

Operator	Meaning	Example
\$eq	Equal to	marks: {\$eq:88}
\$gt	Greater than	marks: {\$gt:85}
\$lt	Less than	marks: {\$lt:60}
\$gte	Greater than or equal to	marks: {\$gte:90}
\$lte	Less than or equal to	marks: {\$lte:70}
\$and	All conditions true	\$and:[...]
\$or	At least one condition true	\$or:[...]

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update, and delete documents. I also learned how to Ascending order & descending order data, comparison operators such as \$eq, \$gt, \$lt,

\$gte, and \$lte work, along with logical operators such as \$and and \$or.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.